AIOps Assignment 3: Spark vs. Ray

Mohammed Khaja Ghouse Mohiuddin (DA24B015)

1. Setup

Both frameworks ran on one laptop (24 logical CPUs, 15.92 GB RAM) as a 2-worker cluster:

- **Spark 3.5 standalone:** one Master (`spark://localhost:7077` , UI :8080) and two Worker daemons. The application requests one executor per worker (cores per executor = cores per worker), so it gets one executor on each worker. Application UI is on :4040.
- **Ray 2.40:** a head node started with `--num-cpus=0` (so it only coordinates) and two worker nodes joined with `ray start --address` . Dashboard is on :8265. Pipeline uses Ray Data.

Data: NYC TLC Yellow Taxi trip records, 3 monthly Parquet files of 2024, plus `taxi_zone_lookup.csv` . The final cleaned table has 8,579,096 rows.

2. Pipeline (identical in both)

Step	Logic
Ingestion	Read all `yellow_tripdata_*.parquet` files and keep 9 columns. Read the zone lookup CSV.
Cleansing	Drop rows with nulls. Keep trip_distance > 0, fare > 0, dropoff after pickup, pickup in 2024. Remove exact duplicate rows. Format timestamps into `pickup_date` (yyyy-MM-dd), `pickup_hour` and `trip_duration_sec` .
Heavy join	Inner join with the zone table twice, on pickup and dropoff location, to get borough and zone names.
UDF	`avg_speed_mph(distance, duration)` , the same Python function from `common.py` in both, applied per row. Trips faster than 100 mph or with invalid duration are dropped.
Feature	Average speed per pickup hour (24 values), joined back onto every trip as `avg_speed_hour` .
Export	Write 18 columns to Parquet.

Implementation difference: Spark uses its built-in `dropDuplicates` , joins and `groupBy` . Ray Data has no global distinct, so rows are hashed into 32 buckets and de-duplicated inside each bucket (identical rows always land in the same bucket). The small zone table is broadcast into each batch with a pandas merge.

3. Pipeline parity

Both outputs were compared by `verify_parity.py` : 8,579,096 vs 8,579,096 rows; order-independent row fingerprint identical; column sums identical; hourly averages differ by at most 1.1e-13 (floating-point summation order only). **Result: PASS.**

4. Performance

Spark: 3 run(s), Ray: 3 run(s); values are mean ± std. dev.

Metric	Spark	Ray
Total wall-clock time (s)	55.68 ± 4.49	177.38 ± 5.69
Peak CPU (%)	87.60 ± 12.55	66.73 ± 11.60

Average CPU (%)	30.83 ± 1.96	22.17 ± 0.85
Peak RAM increase over idle (GB)	5.77 ± 0.16	2.80 ± 0.76
Row-by-row Python UDF (s)	8.64 ± 0.64	14.97 ± 3.05
Arrow pandas UDF (s)	5.03 ± 0.34	n/a
Native / vectorised equivalent (s)	2.59 ± 0.25	7.80 ± 0.67
Python UDF overhead = UDF − native (s)	6.05 ± 0.52	7.17 ± 3.07

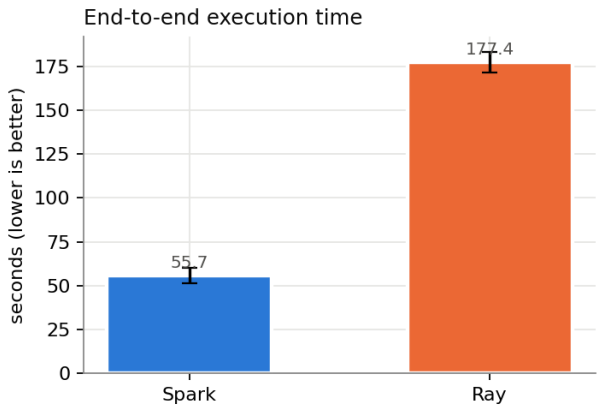


Figure 1: end-to-end time, ingestion to Parquet export

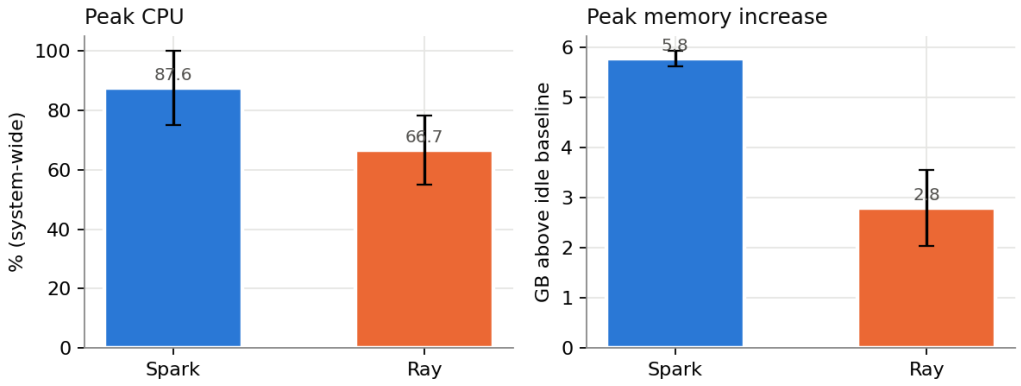


Figure 2: peak CPU and memory (system-wide, sampled every 0.5 s with psutil)

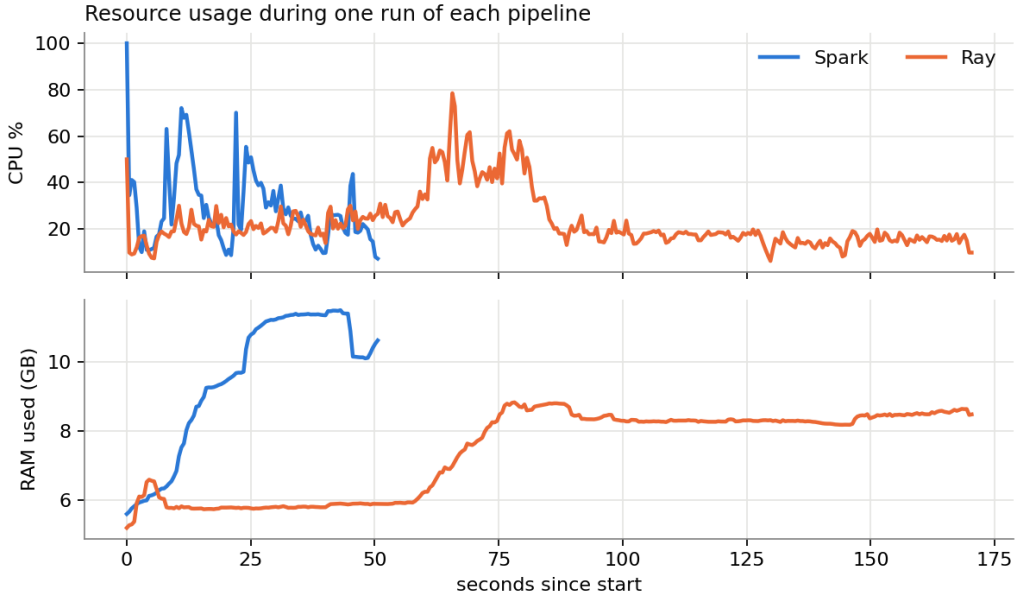


Figure 3: CPU and RAM over time

5. UDF deep-dive: Python-native Ray vs JVM-based Spark

To isolate the UDF cost, each framework re-reads only `trip_distance` and `trip_duration_sec` from the cleaned output (8,579,096 rows) and computes the speed in two ways: with the same row-level Python function used in the pipeline, and with an equivalent native expression (Spark SQL column expression / NumPy vectorised code in Ray). The difference is the overhead of running custom Python code.

- **Spark:** Python UDF 8.64 s vs native 2.59 s, so the UDF is 3.3× slower. Spark executors are JVM processes. A classic Python UDF makes Spark start separate Python worker processes, pickle every row in the JVM, send it over a socket, run the function, and send the result back. That serialisation and copying is the "JVM overhead". Spark's Arrow-based pandas UDF took 5.03 s, which shows how much of the cost is the row-at-a-time serialisation rather than the Python code itself.
- **Ray:** Python UDF 14.97 s vs vectorised 7.80 s, so it is 1.9× slower. Ray workers are Python processes and the data is already pandas/Arrow in Ray's shared-memory object store, so the function runs on the batch directly, with no JVM to Python hop. The remaining cost is just the Python loop itself.

Observation: Ray pays a smaller penalty for custom Python code than Spark (1.9× vs 3.3×). This is the "Python-native" advantage: in Ray, the code you write in Python is the execution engine. In Spark, Python sits outside the engine.

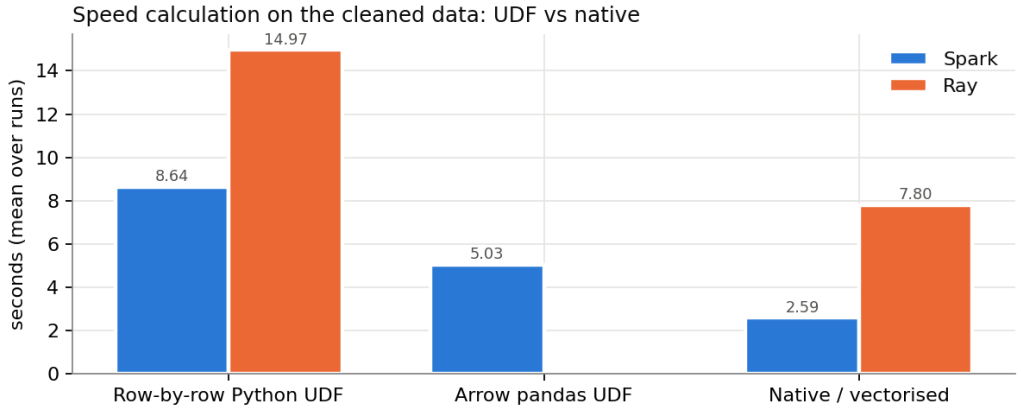


Figure 4: cost of the speed calculation by implementation

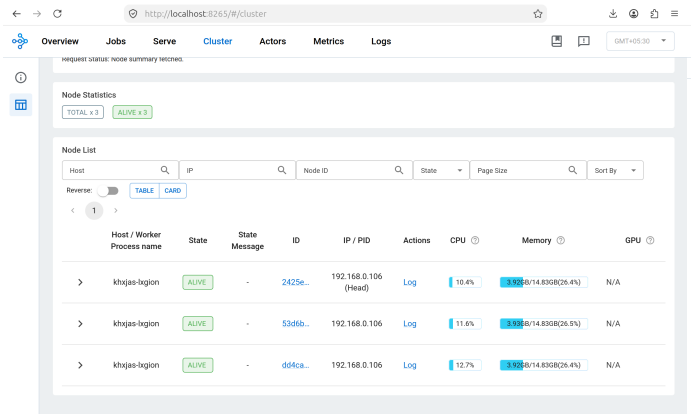
6. Winner

Spark finished the full pipeline faster (55.7 s vs 177.4 s, 3.19×) on this setup.

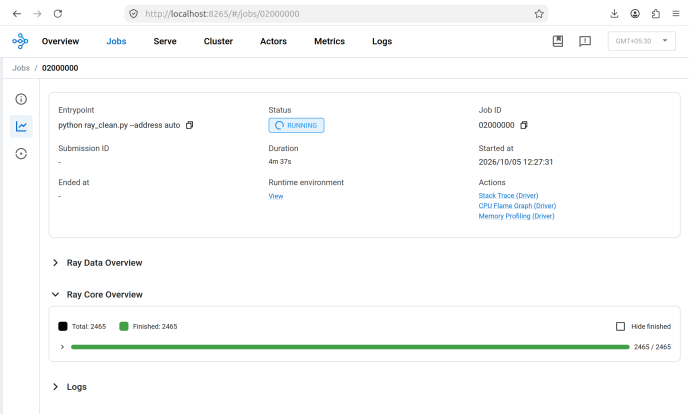
BI-first project (SQL reports, joins, aggregations over structured tables): **Spark**. Its Catalyst optimizer and built-in operators (distinct, joins, groupBy) handle this kind of work natively, it has a SQL interface, and it fits with warehouses and BI tools. The Python UDF penalty can be avoided by writing logic as native expressions.

AI-first project (preprocessing that feeds model training, custom Python, GPUs): **Ray**. Custom Python runs without a JVM hop, the same cluster can also run training, tuning and serving, and operations like distinct that Spark gives for free are easy to write in pandas.

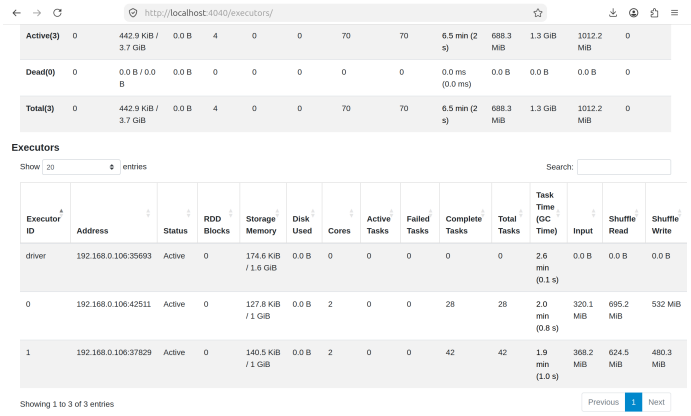
7. Cluster screenshots



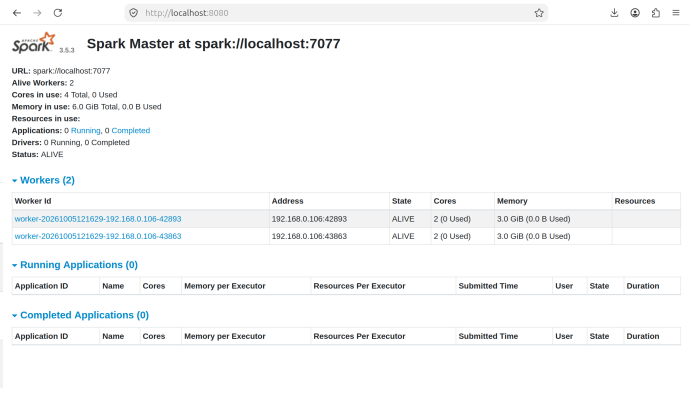
ray dashboard cluster



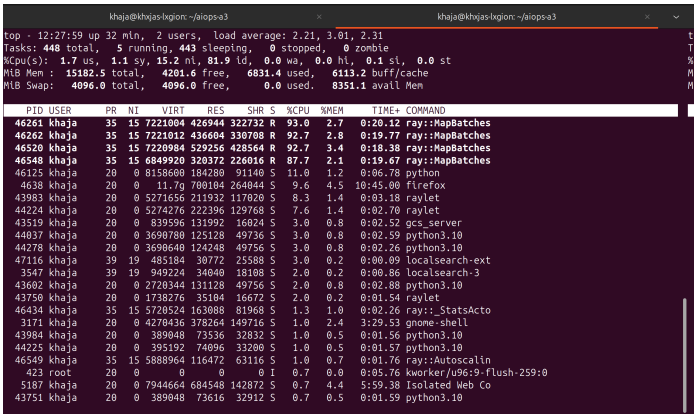
ray dashboard job



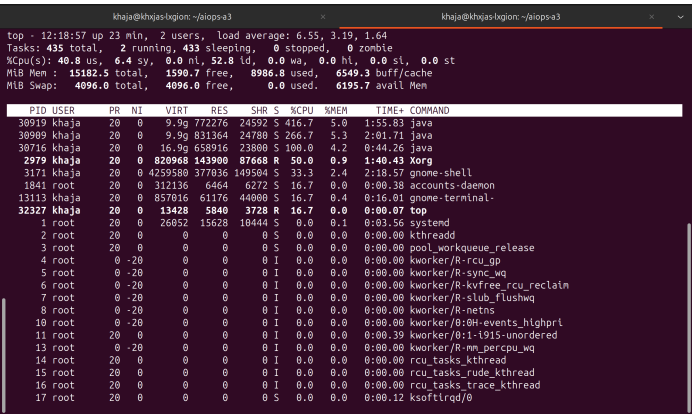
spark app ui executors



spark master ui



top ray



top spark

8. Performance Tuning Note and AI use

An AI assistant (Claude) was used to help translate the pipeline logic between PySpark and Ray Data and to debug it. Its performance suggestions:

- **Ray hourly average:** my first version used `ds.groupby("pickup_hour").mean()`, which sort-shuffles every row and was the slowest step in a test run. Suggestion: compute (sum, count) per hour inside each batch, then combine the small partial table on the driver. Same result, much less data moved.
- **Ray de-duplication:** hash rows into 32 buckets and drop duplicates per bucket, instead of a group per row.
- **Parity settings:** set Spark's session time zone to UTC and write timestamps as microseconds, so both frameworks produce the same timestamps.

The cluster setup was done manually. All numbers in this report come from my own runs, saved in `results/*.json`, and match the Spark UI and Ray Dashboard screenshots.